

Fraud Detection — Full Code Guide

Every file. Every line. Explained.

[Phase 1](#) · [Phase 2](#) · [FastAPI](#) · [Dockerfile](#) · [CI/CD](#)

[train.py](#) · [app.py](#) · [Dockerfile](#) · [ci-cd.yml](#) · [test_api.py](#)

FILE 1 — train.py

What this file does

train.py is the complete ML training pipeline. Run it once to produce all model artifacts (fraud_model.pkl, scalers, threshold). Each function = one pipeline stage. The if `__name__ == '__main__'` block at the bottom calls them in sequence.

IMPORTS — why each library is needed

```
import os, warnings, collections

import numpy as np

import pandas as pd

import joblib

import mlflow, mlflow.xgboost

import matplotlib; matplotlib.use("Agg") # non-interactive backend

import matplotlib.pyplot as plt
```

import os

→ for `os.makedirs` and `os.path.exists` — creating folders and checking file paths

import warnings

→ `warnings.filterwarnings('ignore')` suppresses sklearn deprecation noise in output

import collections

→ `collections.Counter(y_train)` gives clean dict output of class counts

import numpy as np

→ fast numerical arrays. sklearn/XGBoost require numpy arrays as input, not pandas DataFrames

import pandas as pd

→ DataFrame operations — loading CSV, filtering rows, selecting columns

import joblib

→ serialises model objects to .pkl files (faster than pickle for numpy arrays inside sklearn models)

import mlflow

→ experiment tracking — logs every run's params, metrics, and model files

matplotlib.use("Agg")

→ non-interactive backend — required on servers with no display (Docker, CI). Without this, matplotlib crashes trying to open a window

STEP 1 — `load_data()` function

```
def load_data(path="data/creditcard.csv"):

    dtype_map = {f"V{i}": "float32" for i in range(1, 29)}

    dtype_map["Time"] = "float32"
```

```
dtype_map["Amount"] = "float32"

dtype_map["Class"] = "int8"

df = pd.read_csv(path, dtype=dtype_map)

return df
```

dtype_map = {f"V{i}": "float32" ...}

→ dict comprehension — creates {'V1': 'float32', 'V2': 'float32', ...} in one line

"float32" instead of default float64

→ float64 = 8 bytes per value. float32 = 4 bytes. Halves memory for 284k rows x 30 columns = saves ~60MB

"int8" for Class

→ Class is 0 or 1 — fits in int8 (1 byte) vs int64 default (8 bytes). 7 bytes saved per row

pd.read_csv(path, dtype=dtype_map)

→ passing dtypes at load time is faster than converting after — pandas never allocates the larger dtype

STEP 2 — preprocess() — separate scalers for Amount and Time

```
def preprocess(df):
```

```
    X = df.drop("Class", axis=1).copy()

    y = df["Class"].copy()

    amount_scaler = StandardScaler()

    time_scaler = StandardScaler()

    X["Amount"] = amount_scaler.fit_transform(
        X["Amount"].values.reshape(-1, 1))

    X["Time"] = time_scaler.fit_transform(
        X["Time"].values.reshape(-1, 1))

    return X, y, amount_scaler, time_scaler
```

df.drop("Class", axis=1)

→ axis=1 = drop a column. axis=0 would drop a row. .copy() prevents pandas SettingWithCopyWarning

amount_scaler = StandardScaler()

→ separate scaler object for Amount. Using one scaler for both overwrites its saved mean/std on second call — a bug

fit_transform(X['Amount'].values.reshape(-1, 1))

→ reshape(-1,1) converts shape (284807,) to (284807,1). sklearn scalers require 2D input. -1 means 'infer this dimension'

return ..., amount_scaler, time_scaler

→ return both scalers so they can be saved. The API must use the SAME scaler — not refit on new data

STEP 3 — split_data() — stratified train/test

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)
```

test_size=0.2

→ 20% test (56,961 rows), 80% train (227,846 rows)

random_state=42

→ seeds the random number generator — makes the split reproducible. Same data every run

stratify=y

→ ensures both sets have 0.17% fraud. Without it, random chance might put all 492 fraud cases in one set

STEP 4 — apply_smote() — only on training data

```
smote = SMOTE(random_state=42, k_neighbors=5)  
  
X_sm, y_sm = smote.fit_resample(X_train, y_train)  
  
# Before: {0: 227451, 1: 394}  
# After: {0: 227451, 1: 227451}
```

SMOTE(k_neighbors=5)

→ for each fraud sample, finds 5 nearest fraud neighbours and creates new samples between them

fit_resample(X_train, y_train)

→ X_train only — NEVER X_test. Test set must remain real data for honest evaluation

Result: equal class sizes

→ SMOTE duplicates fraud from 394 to 227,451 synthetically — balanced training set

STEP 5 — train_xgboost() — production model

```
neg = int((y_train == 0).sum())  
  
pos = int((y_train == 1).sum())  
  
scale_weight = neg / pos # 577.0  
  
xgb = XGBClassifier(  
    n_estimators=1000,  
    max_depth=6,  
    learning_rate=0.05,  
    subsample=0.8,
```

```

colsample_bytree=0.8,
scale_pos_weight=scale_weight,
eval_metric="aucpr",
early_stopping_rounds=50,
n_jobs=-1,
)
xgb.fit(X_train, y_train,
eval_set=[(X_test, y_test)],
verbose=100)

```

scale_weight = neg / pos

→ 577.0 — each fraud sample contributes 577x more to the gradient update. Key imbalance fix for XGBoost

n_estimators=1000

→ set high — early_stopping_rounds will stop training automatically when it plateaus

early_stopping_rounds=50

→ stops if AUC-PR on eval_set doesn't improve for 50 consecutive rounds. Prevents overfitting, finds optimal n_estimators

subsample=0.8, colsample_bytree=0.8

→ row and column subsampling — regularisation to prevent overfitting

eval_metric='aucpr'

→ XGBoost optimises AUC-PR (not accuracy). Correct for imbalanced fraud data

n_jobs=-1

→ uses all available CPU cores for parallel tree building

STEP 6 — tune_threshold() — F2-beta optimisation

```
precisions, recalls, thresholds = precision_recall_curve(
```

```
y_test, y_prob_xgb)
```

```
beta = 2
```

```
f2 = ((1 + beta**2) * precisions * recalls /
```

```
(beta**2 * precisions + recalls + 1e-9))
```

```
best_idx = np.argmax(f2[:-1])
```

```
best_threshold = thresholds[best_idx]
```

precision_recall_curve()

→ returns precision, recall, threshold arrays at every possible cutoff

beta = 2

→ F2: recall is 2x more important than precision. Correct when FN (missed fraud) costs more than FP (false alarm)

1e-9 in denominator

→ tiny constant prevents ZeroDivisionError when precision + recall = 0 at extreme thresholds

f2[:-1]

→ precision_recall_curve returns n+1 precision/recall values but only n thresholds. [:-1] aligns the lengths

thresholds[best_idx]

→ the actual threshold value (e.g. 0.31) that maximises F2. Use this in the API instead of default 0.5

STEP 7 — save_artifacts() — save everything the API needs

```
joblib.dump(xgb, "models/fraud_model.pkl")
```

```
joblib.dump(amount_scaler, "models/amount_scaler.pkl")
```

```
joblib.dump(time_scaler, "models/time_scaler.pkl")
```

```
joblib.dump(best_threshold, "models/threshold.pkl")
```

joblib.dump()

→ serialises Python objects to disk. Faster than pickle for numpy arrays

separate amount and time scalers

→ we need BOTH to inverse-transform in the API. One combined scaler would lose the ability to scale them independently

threshold.pkl

→ saves the float threshold value. API loads it at startup so threshold is not hardcoded

FILE 2 — app.py (FastAPI Endpoint)

What this file does

app.py is the production REST API. It loads the trained model once at startup and serves predictions at /predict. A payment gateway calls this endpoint when a transaction arrives. Response must come back in under 100ms.

Import section — what each import adds

```
from fastapi import FastAPI, HTTPException

from pydantic import BaseModel, validator

from contextlib import asynccontextmanager

import joblib, numpy as np, time, logging
```

FastAPI

→ the web framework. Auto-generates OpenAPI docs at /docs

HTTPException

→ raises HTTP error responses (e.g. 500) with custom messages

BaseModel, validator

→ Pydantic — validates incoming JSON. Wrong types return 422 before reaching model code

asynccontextmanager

→ enables the lifespan startup/shutdown pattern

time

→ time.perf_counter() for high-resolution millisecond latency measurement

logging

→ structured logs to stdout — readable by CloudWatch/Datadog in production

model_store and lifespan pattern

```
model_store: dict = {}

@asynccontextmanager
async def lifespan(app: FastAPI):

    model_store["model"] = joblib.load(MODEL_PATH)

    model_store["threshold"] = joblib.load(THRESH_PATH)

    model_store["amount_scaler"] = joblib.load(AMT_SCALER_PATH)

    model_store["time_scaler"] = joblib.load(TIME_SCALER_PATH)

    model_store["request_count"] = 0

    yield

    model_store.clear()
```

model_store: dict = {}

→ module-level dictionary — lives in RAM for the entire server lifetime. All workers access it

joblib.load() BEFORE yield

→ loads model into RAM once at startup. Subsequent predictions read from memory = microseconds, not 500ms disk reads

yield

→ FastAPI serves all requests between startup and shutdown here. Everything before yield = startup, after = shutdown

model_store.clear()

→ cleanup at shutdown — releases memory

TransactionRequest — Pydantic validation

```
class TransactionRequest(BaseModel):

    transaction_id: str

    features: list[float]

    @validator("features")
    def validate_feature_count(cls, v):
        if len(v) != 30:
            raise ValueError(
                f"Expected 30 features, got {len(v)}")
        return v
```

BaseModel

→ all fields are type-validated automatically by FastAPI before the endpoint function runs

list[float]

→ expects a JSON array of numbers. FastAPI will reject strings or missing fields

@validator('features')

→ custom validation that runs AFTER type checking. Ensures exactly 30 features

raise ValueError

→ Pydantic catches this and returns a 422 response with clear error detail to the client

preprocess_transaction() — apply saved scalers

```
def preprocess_transaction(features):

    X = np.array(features, dtype=np.float32).reshape(1, -1)

    X[0, 0] = model_store["time_scaler"].transform(
        [[X[0, 0]])[0, 0]

    X[0, 29] = model_store["amount_scaler"].transform(
        [[X[0, 29]])[0, 0]
```

return X

reshape(1, -1)

→ converts flat list to shape (1, 30) — model expects 2D input. 1 sample, 30 features

time_scaler.transform()

→ apply saved training-time mean/std. NOT fit_transform — that would compute new stats from one sample

X[0, 0]

→ index 0 = Time feature (first column). X[0, 29] = Amount (last column). V1-V28 (indices 1-28) are already scaled

[0, 0] after transform

→ transform returns shape (1,1). [0,0] extracts the single float value

POST /predict endpoint

```
@app.post("/predict", response_model=PredictionResponse)
```

```
def predict(tx: TransactionRequest):
```

```
    start = time.perf_counter()
```

```
    X = preprocess_transaction(tx.features)
```

```
    prob = float(model_store["model"].predict_proba(X)[0,1])
```

```
    is_fraud = prob >= model_store["threshold"]
```

```
    risk = get_risk_level(prob, model_store["threshold"])
```

```
    ms = (time.perf_counter() - start) * 1000
```

```
    return PredictionResponse(
```

```
        fraud_probability=round(prob, 4),
```

```
        is_fraud=is_fraud, risk_level=risk,
```

```
        decision_ms=round(ms, 2), ...)
```

def predict (not async def)

→ sync function: FastAPI runs in a thread pool. Correct for CPU-bound inference. async def blocks the event loop

time.perf_counter()

→ nanosecond-resolution timer. More accurate than time.time() for sub-millisecond measurement

predict_proba(X)[0,1]

→ [0] = first sample, [1] = probability of class 1 (fraud). Column 0 would be P(legitimate)

float()

→ converts numpy float32 to Python float — required for JSON serialisation

response_model=PredictionResponse

→ FastAPI validates and serialises the return dict against this schema

FILE 3 — Dockerfile

What this file does

The Dockerfile packages your app, model, and all dependencies into a container image. The same image runs identically on your laptop, GitHub Actions, and any cloud server.

```
FROM python:3.11-slim
```

→ base image: Python 3.11 without build tools. 'slim' reduces image from ~1GB to ~150MB

```
WORKDIR /app
```

→ all subsequent commands run inside /app inside the container

```
COPY requirements.txt .
```

→ copy requirements FIRST — Docker caches this layer. If only code changes, pip install is not re-run

```
RUN pip install --no-cache-dir -r requirements.txt
```

→ --no-cache-dir: don't store pip's download cache = saves ~50MB in final image

```
COPY app.py . / COPY models/ models/
```

→ copy only runtime files. Never copy training data, notebooks, or .git

```
RUN useradd -m -u 1000 appuser / USER appuser
```

→ run as non-root. Root inside container = security vulnerability

```
EXPOSE 8000
```

→ documents the port — does NOT open it. Actual mapping: docker run -p 8000:8000

```
HEALTHCHECK ...
```

→ Docker will restart the container if health check fails 3 times. Ensures self-healing

```
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", ...]
```

→ array form avoids shell interpretation. host 0.0.0.0 required inside container

FILE 4 — .github/workflows/ci-cd.yml

What this file does

This GitHub Actions workflow automatically runs tests and builds Docker on every push. Interviewers see a green checkmark badge on your repo — shows production engineering mindset.

```
on: push: branches: [main]
```

→ triggers the pipeline on every git push to main branch

```
on: pull_request: branches: [main]
```

→ also triggers on any PR targeting main — catches bugs before merge

```
runs-on: ubuntu-latest
```

→ GitHub hosts a fresh Ubuntu VM for each run. Free for public repos

```
uses: actions/checkout@v4
```

→ checks out your repository code into the runner VM

```
uses: actions/setup-python@v5 / cache: 'pip'
```

→ installs Python 3.11 and caches pip packages between runs (faster builds)

```
pip install -r requirements.txt
```

→ installs all dependencies — same as local dev

```
pytest tests/ -v --tb=short
```

→ runs all tests. Pipeline fails (red x) if any test fails

```
needs: test
```

→ build job only runs if test job passes. Sequential dependency

```
docker build -t fraud-api:${{ github.sha }}
```

→ tags image with git commit hash — unique, traceable image per commit

```
secrets.DOCKER_USERNAME / DOCKER_PASSWORD
```

→ GitHub encrypted secrets — never hardcode credentials in YAML files

```
if: github.event_name == 'push'
```

→ push to Docker Hub only on main pushes, not on PRs

FILE 5 — tests/test_api.py

What this file does

Automated tests for the FastAPI endpoint. These run in CI without needing real .pkl model files — they use mocks. Every test corresponds to one thing that could go wrong in production.

```
from unittest.mock import MagicMock, patch
```

→ mock = fake object that replaces a real object. patch() swaps model_store with a fake during tests

```
TestClient(app)
```

→ FastAPI's built-in test client — makes real HTTP requests to the app without starting a server

```
@pytest.fixture
```

→ marks a function that sets up test data/state. pytest injects it into any test that lists it as a parameter

```
def test_health_returns_200
```

→ test names must start with 'test_'. pytest discovers them automatically

```
assert response.status_code == 200
```

→ assert = if this is False, the test fails with a clear message

```
def test_predict_rejects_wrong_feature_count
```

→ tests what happens with bad input — as important as testing happy path

```
continue-on-error: true (in CI)
```

→ lint warnings don't fail the pipeline — only test failures do

CODE EFFICIENCY — 6 Bugs Found & Fixed

BUG 1 — One scaler used for both Amount and Time

Problem: `scaler.fit_transform(Amount)` then `scaler.fit_transform(Time)` — second call OVERWRITES first. Saved scaler has Time's stats, not Amount's. API cannot correctly inverse-transform Amount.

Fix: Use separate `StandardScaler()` objects: `amount_scaler` and `time_scaler`. Save both separately.

BUG 2 — SMOTE applied before train/test split

Problem: Synthetic fraud samples derived from test transactions appear in training set. Test metrics measure performance on fake data. All evaluation numbers are dishonest.

Fix: Always: (1) split, (2) SMOTE only on `X_train`. Test set must stay real data.

BUG 3 — Correlation computed on full dataset

Problem: `df.corr()` uses all 284k rows including test data. Feature selection biased toward test set patterns — subtle leakage.

Fix: Compute correlation on training DataFrame only: `pd.concat([X_train, y_train], axis=1).corr()`

BUG 4 — No early stopping in XGBoost

Problem: Fixed `n_estimators=300` might overfit (too many) or underfit (too few). No way to know the optimal tree count.

Fix: Set `n_estimators=1000`, `early_stopping_rounds=50`, `eval_set=[(X_test, y_test)]`. XGBoost finds optimal automatically.

BUG 5 — Model loaded per request in FastAPI

Problem: `joblib.load()` inside the predict function = 200-500ms disk read on EVERY call. Payment gateway SLA is 100ms total. Every request times out.

Fix: Load model ONCE in lifespan startup. Store in `model_store` dict. All requests read from RAM = microseconds.

BUG 6 — SMOTE + scale_pos_weight used together

Problem: SMOTE balances classes (1:1 ratio). Then `scale_pos_weight=577` up-weights fraud again. Model becomes excessively aggressive — precision drops below 50%.

Fix: Use one or the other. SMOTE for LogisticRegression. `scale_pos_weight` for XGBoost. Never both simultaneously.